

BYTES ARE NOT LATENCY: ALLOCATED SIZE, MEASURED TRAFFIC, DECODE SPEED, AND QUALITY OF KV-CACHE QUANTIZATION ON A SINGLE BLACKWELL WORKSTATION GPU

Anonymous Authors¹

ABSTRACT

KV-cache quantization is expected to speed up decoding because a memory-bound decode step should shorten when it reads fewer bytes. We test this expectation, together with model quality, for TurboQuant, KIVI, and KVQuant in one full-model Llama-3.1-8B-Instruct decode harness on an NVIDIA RTX PRO 6000 Blackwell GPU, and find that in these implementations fewer bytes do not mean lower latency: every compressed configuration moves fewer cache bytes than BF16, yet none is faster than BF16 FlashAttention at any same-work point. What decides latency is how efficiently the kernels move bytes, not how many they move. KIVI, the cleanest comparison because its cache-path kernels are the official ones, reads $2.8\text{--}4.3\times$ fewer cache bytes than BF16 but runs its cache path at 8–13% of peak DRAM bandwidth at batch size 1, against 89% for BF16; even without our adapter’s fixed overhead, KIVI at best matches BF16. The TurboQuant and KVQuant ports are slower still, and in all three methods large, identifiable costs trace to integration choices rather than to the algorithms: a split-KV setting below vLLM’s default that leaves most of the GPU idle, a single-threaded outlier-selection kernel that we introduced to make KVQuant graph-safe and deterministic, and per-query-head residual kernels in our KIVI adapter; our correctness gates caught none of them. Modeled bandwidth ceilings bound what better kernels could gain, and under a cache-sensitive quality protocol for single-sequence decoding no configuration is both quality-qualified and faster than BF16.

1 RESEARCH QUESTIONS AND MOTIVATION

During autoregressive decoding every new token attends over the full key–value (KV) cache, so at long context the cache dominates both device memory and per-step memory reads. For Llama-3.1-8B-Instruct [13], one 128K-token sequence holds 17.2 GB of BF16 keys and values. KV-cache quantization stores this state in a few bits per element [10, 15–17, 21, 30, 32, 35–37] and is expected to deliver two benefits: capacity, because more tokens fit in memory, and speed, because a bandwidth-bound decode step should shorten when it reads fewer bytes.

The speed argument rests on a chain of proxies—nominal bit width, allocated bytes, physical DRAM traffic, latency—and each link can break. Allocated storage includes scales, norms, full-precision windows, sparse outliers, and workspace; traffic depends on what kernels actually read; latency depends on how much parallelism kernels expose and how many kernels the host submits. A quality claim

further requires that the model behaves well when it must read its compressed cache. Published evaluations measure these links with each method’s own harness and hardware, and a serving-oriented re-examination notes that KV-cache compression remains uncommon in production [12]; the exception is FP8 KV caching, which serving engines such as vLLM and TensorRT-LLM support [27, 31].

We study three methods that stress different links. TurboQuant rotates vectors and applies scalar codebooks, giving a dense packed layout of fixed size [37]. KIVI quantizes keys per channel and values per token and keeps a recent window in full precision, so its effective ratio varies with context [21]. KVQuant quantizes pre-RoPE keys with sensitivity-weighted non-uniform datatypes, stores outliers sparsely, and keeps the first tokens in FP16 as attention sinks [16, 33]. We ask four questions:

- RQ1 (bytes). How do nominal bit width, allocated cache bytes, and measured DRAM traffic relate for each method, and does any validated path materialize grouped-query K/V at query-head width?
- RQ2 (latency). Does a smaller cache yield same-work decode speedup on this GPU, and is the sensitivity of the latency curve to CUDA Graphs explained by

¹ Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

launch overhead alone?

- RQ3 (prediction). Is a method-conditioned function of batch size B , context length L , and allocated ratio r sufficient to predict decode latency for held-out batches, configurations, and context bands?
- RQ4 (quality). Which compressed configurations preserve quality when evaluation reads the compressed cache, and what survives a joint quality–performance validation?

Findings. RQ1: allocated compression falls short of nominal by a method-specific margin, measured cache traffic departs from the allocation in both directions, and no validated path materializes K/V at query-head width. RQ2: no compressed configuration is faster than BF16 at any of the 357 same-work points (TurboQuant and KVQuant as ported), and their cache-path kernels run far below BF16’s bandwidth; CUDA Graphs remove submission overhead for every method but reshape the latency curves in method-specific ways, so the floor is not launch overhead alone. RQ3: a method-conditioned knee surface in B , L , and r_{alloc} predicts held-out latency far better than a scalar byte law (7.9% against 34.6% median error) but misses all four evaluable targets. RQ4: at physical batch size 1, only KIVI-k4v4 passes perplexity and LongBench-E; it then fails two small-count LongBench v2 guardrails, and KVQuant’s result is invalid, so no configuration is both quality-qualified and faster than BF16.

Related work. Most KV-cache quantizers refine how bits are spent across tokens and channels: importance-aware mixed precision [35], salient-token identification [15], sliding-window quantization [10], near-lossless compression recipes [17], subspace-orthogonal quantization [32], hardware-aware tuning-free quantization [30], and 1-bit quantized Johnson–Lindenstrauss transforms [36]. These papers report accuracy and efficiency in their own harnesses; we measure three of them with model, hardware, timing boundary, and evaluation held fixed. Rotation-based outlier suppression [2, 11] is related to the Hadamard rotation that TurboQuant’s vLLM implementation applies to keys. Low-rank projection [6] compresses the hidden dimension of cached keys and values, an axis orthogonal to bit width that we leave aside. Eviction and selection methods [5, 7, 9, 19, 34, 38] shrink the cache along the token axis; KVQuant’s FP16 sink tokens follow the attention-sink observation of [33]. Serving-oriented work compresses KV caches for streaming and transfer between machines [20, 22], and Gao et al. [12] revisit compression techniques from the perspective of production serving. We complement these with controlled same-work decode timing on one GPU.

Table 1. Configurations and implementation sources.

Family	Configurations (key/value bits)	Implementation source	Project changes
BF16	BF16	PyTorch SDPA, FlashAttention backend	none
TurboQuant	TQ-4bit (4/4), TQ-k3v4 (3/4), TQ-3bit (3/3)	TurboQuant KV-cache path of vLLM v0.25.1	port into the common harness
KIVI	k4v4, k2v4, k2v2; group 32, residual 32	official repository, post-paper snapshot	measured: Graph-safe adapter, per-head residual kernels, FP16 staging; GQA source patch for fixtures
KVQuant	KVQuant-4, -3, -2; 5 sink tokens; outlier cap 12	author repository	GQA/RoPE compatibility; graph-safe, deterministic kernels

2 EXPERIMENTAL SCOPE AND MEASUREMENT METHODOLOGY

2.1 System under test

All experiments use Llama-3.1-8B-Instruct at one fixed revision with BF16 weights: 32 layers, 32 query heads sharing 8 KV heads through grouped-query attention [1], and head dimension 128. The hardware is one NVIDIA RTX PRO 6000 Blackwell GPU (96 GB, compute capability 12.0) with tensor parallelism 1. All timing runs execute in one digest-pinned container with PyTorch 2.12.1 and CUDA 13.0 [28]. The BF16 baseline calls PyTorch scaled-dot-product attention restricted to its FlashAttention backend with native GQA [8]; an unsupported shape raises an error. Caches are static, preallocated, and stored at eight KV heads; `torch.compile` is disabled.

2.2 Methods and implementation provenance

Every compressed method runs from adapted code (Table 1).

TurboQuant. The paper names no author implementation, so we use the TurboQuant path of vLLM v0.25.1 [18]. It applies a Hadamard rotation and Lloyd–Max scalar quantization to keys with a per-vector FP16 norm, quantizes values uniformly with FP16 scale and zero point, omits the paper’s QJL stage [36], and keeps the first two and last two attention layers in BF16. Our port fixes the decode kernel’s split-KV count at 4, whereas vLLM v0.25.1 uses 32 by default. We report TurboQuant as ported and write TurboQuant[†] (as

ported, split = 4) wherever its ratios or latencies appear; with 32 splits, a post-hoc Nsight Systems diagnostic still measures $1.6\text{--}3.2\times$ BF16’s GPU kernel time, which is not wall-clock time (Section 3.3).

KIVI. We use the official repository at a post-paper snapshot that advertises Llama-3/GQA support, with group size 32 and a 32-token full-precision residual window (the paper’s main experiments mostly use 128; its appendix reports 32). The snapshot’s residual-window attention expands K/V from 8 to 32 heads (Section 3.5). A source patch computes both residual contractions as batched products grouped by KV head; it matches the original formula exactly in BF16 checks, leaves quantization, packing, metadata, and rollover unchanged, and generates the G1 reference fixtures, but is not on the measured path. Performance and quality runs both use our Graph-safe adapter, which calls the official kernels for the quantized history and computes the residual window and the current token per query head (96 FP16 bmm and 64 add kernels per layer in the traced step, Section 4). The official CUDA kernels accept only FP16, so BF16 activations are staged through preallocated FP16 buffers.

KVQuant. We use the author repository with a compatibility patch (“KVQuant-GQA patched upstream”), because the pinned revision rejects Llama-3.1 RoPE and native GQA. The patch adds both, plus a project-defined sparse-outlier capacity of 12 entries per token row (six per tail of the 1,024-wide KV row), deterministic tie-breaking, and five FP16 sink tokens; the harness supplies pre-RoPE keys. To pass our allocation, determinism, and graph gates, we also replaced dynamic allocation and host synchronization in sparse selection and value packing with fixed caller-owned buffers, replaced floating-point atomics in long-context value decode with per-tile partials and an ordered reduction, and initialized shared-memory lanes that the original value kernels read uninitialized. Quantizers, packing, and sparse semantics are unchanged. Calibration uses sixteen 2,048-token WikiText-2 training windows [23] with Fisher-weighted non-uniform quantizers.

2.3 Decode endpoint and timing

The primary runner fixes the historical context L , builds the cache outside timing, and replays one decode step as a CUDA Graph at unchanged shape. A step embeds one token per sequence, runs all 32 layers against the cache, and applies the final norm and LM head to produce full-vocabulary logits. Sampling is excluded, and the measured region contains no allocation, concatenation, host synchronization, or tensor-to-host conversion. Latency is the wall-clock time of one full-batch decode step, observed on the host. Each process runs 64 warmup and 256 measured steps. A point’s latency is the median of five process medians, and points whose cross-process coefficient of variation (CV) exceeds

3% are marked unstable. CUDA-event time serves as a secondary diagnostic. Points run in blocked randomized order, with block order rotated across the five process repetitions. We write the largest context, 131,071 historical tokens plus the current one, as 128K.

2.4 Validation and scan

A configuration entered the scan only after passing five shared gates at $B = 1$, $L = 4,096$, Graph mode: agreement with its own method’s reference implementation (G1); predicted and allocated bytes within 1% (G2); a clean execution path, with no dequantized full-precision copy of the cache prefix, GQA repeat materialization, measured-region allocation or host synchronization, or backend fall-back (G3); correct graph replay without allocation (G4); and three-process repeatability (G5). Each method’s kernels also passed reference-fixture, Compute Sanitizer, and allocation tests. All ten configurations passed.

A three-process pilot located provisional knees. Because knee density was insufficient for 25 of 30 curves, a separately preregistered densification step added 84 contexts before the full scan. The full scan covers $B \in \{1, 2, 4, 8, 16\}$ and nine base contexts from 4K to 128K for all ten configurations, plus the 84 adaptive points: 534 points. Feasibility was predicted before launch against 88% of device memory; 441 points are feasible and 93 are recorded as capacity-infeasible. All 441 feasible points were measured in five processes each and are stable (maximum CV 1.31%); Appendix A gives the run accounting.

2.5 Bytes, traffic, and comparison types

We separate three compression ratios. The nominal ratio is

$$r_{\text{nom}} = \left[f + (1 - f) \frac{q_K + q_V}{32} \right]^{-1},$$

where q_K and q_V are the configured key and value bit widths and f is the fraction of layers kept in BF16 (4/32 for TurboQuant, 0 otherwise). The allocated ratio is

$$r_{\text{alloc}} = \frac{C_{\text{logical}}}{C_{\text{alloc}}},$$

where C_{logical} is the logical BF16 K/V payload at (B, L) and C_{alloc} is the storage a configuration actually allocates, including metadata, residual, sink, outlier, padding, and workspace bytes; for BF16 itself r_{alloc} is slightly below 1 because of a small workspace. The measured ratio is

$$r_{\text{DRAM}} = \frac{D_{\text{BF16}}}{D_{\text{method}}},$$

where both are DRAM bytes (GDDR7 on this GPU) that Nsight Compute [25] measures for cache-path kernels;

Table 2. Nominal and allocated compression at $B = 1$, and the main sources of compression shortfall at 128K (logical BF16 K/V payload: 17.18 GB).

Configuration	r_{nom}	$r_{\text{alloc, 4K}}$	$r_{\text{alloc, 128K}}$	Main sources of compression shortfall at 128K (GB)
TQ-4bit	2.91	2.22	2.23	BF16 boundary layers 2.15; decode workspace 1.62
TQ-k3v4	3.16	2.37	2.38	same as TQ-4bit
TQ-3bit	3.46	2.53	2.54	same as TQ-4bit
KIVI-k4v4	4.00	3.04	3.14	scale/zero metadata 1.07
KIVI-k2v4	5.33	3.75	3.90	scale/zero metadata 1.07
KIVI-k2v2	8.00	4.88	5.16	scale/zero metadata 1.07
KVQuant-4	4.00	3.08	3.14	sparse outliers 0.81; metadata 0.30
KVQuant-3	5.33	3.98	4.05	sparse outliers 0.81; metadata 0.17
KVQuant-2	8.00	5.44	5.54	sparse outliers 0.81; metadata 0.10

r_{DRAM} exists only where a profiler measured it, and timing analysis excludes profiler durations. Traffic amplification is

$$A = \frac{D_{\text{method}}/D_{\text{BF16}}}{C_{\text{alloc, method}}/C_{\text{alloc, BF16}}},$$

computed against BF16’s actual allocation; A exceeds 1 when a method moves more bytes than its allocation implies.

A same-work ratio $S = T_{\text{BF16}}/T_{\text{method}}$ compares identical B , L , output work, graph mode, and timing boundary; $S < 1$ means the method is slower. A capacity-amplification point is one where the method is feasible and BF16 is not. It has no latency denominator, and we report the two comparison types separately.

3 ALLOCATED BYTES, MEASURED TRAFFIC, AND DECODE BEHAVIOR

Results in this section describe these implementations on this GPU, not the quantization algorithms in general.

3.1 Nominal versus allocated compression

Allocated compression is below nominal for every configuration, and both the gap and its dependence on context are method-specific (Table 2). TurboQuant’s ratio is nearly flat in L because its BF16 boundary layers and decode workspace both scale with context. KIVI’s ratio rises with context as the fixed 32-token window amortizes, but its per-group scale and zero-point metadata alone occupy 1.07 GB at 128K. KVQuant’s sparse outlier storage has fixed

capacity per token, so it is identical across bit widths (0.81 GB at 128K) and weighs more as the dense payload shrinks. Nominal bit width therefore overstates compression by a configuration-dependent margin and cannot stand in for allocated bytes.

3.2 Measured DRAM traffic

Nsight Compute profiled all ten configurations at one common geometry: $B = 1$, 128K, Graph mode (Table 3).

Allocation and traffic disagree in both directions. TurboQuant moves fewer bytes than its allocation implies ($A = 0.76$ – 0.80), consistent with its 1.62 GB decode workspace being allocated but not read, while KIVI moves more ($A = 1.12$ – 1.20); the available KIVI kernel symbols do not isolate residual-copy traffic, so KIVI’s excess remains unattributed. KVQuant’s frozen values ($A = 0.82$ – 1.01) leave its key-score and sparse-key kernels outside the cache path; counting them gives $A = 1.46$ – 1.79 (post hoc, Table A2).

Every compressed configuration transfers 2.8 – $5.5\times$ fewer cache bytes than BF16 (2.2 – $4.3\times$ with KVQuant’s key kernels counted) and 20.3 – 25.2 GB in total against BF16’s 32.5 GB, yet all are slower ($S = 0.048$ – 0.499 , lowest for TurboQuant[†]). The difference lies in how fast the bytes move: post hoc, BF16’s cache path runs at 89% of peak DRAM bandwidth, KIVI’s at 8–13%, KVQuant’s at 1–2% (2–4% with its key kernels counted), and TurboQuant[†]’s at about 1%. In these implementations cache traffic volume does not determine latency; the efficiency of the kernels that move it does.

To bound what better kernels could gain, we model each configuration’s DRAM traffic at every (B, L) from its allocated bytes and the profiled points, and compute two post-hoc ratios (Appendix C): $S_{\text{eq}} = D_{\text{BF16}}/D_{\text{method}}$, the same-work ratio if the method moved its own traffic as efficiently as BF16, and the cache-only ceiling

$$S_{\text{cache}} = \frac{T_{\text{BF16}}}{T_{\text{nc}} + D_{\text{ideal, cache}}/BW_{\text{peak}}},$$

which keeps BF16’s measured non-cache time $T_{\text{nc}} = T_{\text{BF16}} - D_{\text{cache, BF16}}/BW_{\text{eff, BF16}}$, with $BW_{\text{eff, BF16}}$ BF16’s fitted cache bandwidth (Section 4), and replaces only the cache path with an ideal one that reads the method’s allocation once at peak bandwidth. At the 357 same-work points S_{eq} is 1.00–1.96 and S_{cache} 1.02–2.17, and no measured ratio exceeds either; measured S reaches 28–65% of S_{cache} for KIVI, 3–58% for TurboQuant[†], and 0.6–8% for KVQuant (Figure 1b). Where a batch holds fewer than 16K tokens, both ratios stay below 1.05 (at $B = 1$, 4K the cache path is 2.9% of BF16’s step time), so $S < 1$ at those points says nothing about the value of byte savings. Within a family, latency also responds weakly to bytes: across

Table 3. Profiled DRAM traffic at $B = 1, 128K$, Graph mode (Nsight Compute: cold L2 for every kernel replay, base clocks) and the same-work ratio from normal timing at the same point; these frozen columns are regenerated by the joint-results package (Appendix A). Post hoc: effective bandwidth as a share of the 1,792 GB/s peak, total decode DRAM over the step time, and the cache-path DRAM added from 4K to 128K over the latency added. [†]As ported, split = 4 (split = 32 diagnostic: 1.6–3.2× BF16’s Nsight Systems GPU kernel time, not wall-clock; Appendix D). *The frozen kernel classification counts KVQuant’s key-score and sparse-key kernels as other model traffic; Table A2 moves them into the cache path (post hoc).

Configuration	Frozen						Post hoc: BW (% of peak)	
	Cache-path DRAM (GB)	Total DRAM (GB)	r_{alloc}	r_{DRAM}	A	S	Total	Cache path
BF16	17.38	32.50	1.00	1.00	1.00	—	81.4	89.3
TQ-4bit [†]	6.25	21.38	2.23	2.78	0.80	0.048	2.6	0.8
TQ-k3v4 [†]	5.75	20.88	2.38	3.02	0.79	0.072	3.8	1.1
TQ-3bit [†]	5.19	20.32	2.54	3.35	0.76	0.060	3.1	0.8
KIVI-k4v4	6.18	23.08	3.14	2.81	1.12	0.486	28.1	12.5
KIVI-k2v4	5.10	22.01	3.90	3.41	1.15	0.499	27.5	10.8
KIVI-k2v2	4.03	20.94	5.16	4.31	1.20	0.471	24.7	7.7
KVQuant-4*	4.54	25.21	3.14	3.82	0.82	0.081	5.1	2.0
KVQuant-3*	3.78	23.61	4.05	4.60	0.88	0.081	4.8	1.7
KVQuant-2*	3.16	22.45	5.54	5.50	1.01	0.081	4.5	1.4

Table 4. Wall-clock latency per full-batch decode step (ms), median of five process medians. [†]As ported, split = 4 (split = 32 diagnostic: 1.6–3.2× BF16’s Nsight Systems GPU kernel time, not wall-clock; Appendix D).

Configuration	$B = 1, 4K$	$B = 1, 32K$	$B = 1, 128K$	$B = 8, 4K$	$B = 8, 32K$	$B = 16, 4K$
BF16	11.77	14.12	22.29	14.28	32.67	17.30
TQ-4bit [†]	25.83	123.92	460.22	26.02	122.72	27.16
TQ-k3v4 [†]	21.09	86.82	309.89	22.02	91.10	24.65
TQ-3bit [†]	22.88	101.21	369.84	23.52	103.14	25.66
KIVI-k4v4	19.06	24.53	45.83	21.06	43.23	24.47
KIVI-k2v4	19.08	24.34	44.63	20.84	40.51	23.88
KIVI-k2v2	19.10	24.99	47.31	20.68	38.83	23.63
KVQuant-4	151.08	154.41	273.86	1,077.92	1,172.11	2,153.59
KVQuant-3	152.12	158.56	275.05	1,078.92	1,186.79	2,173.71
KVQuant-2	149.27	169.47	275.64	1,065.92	1,232.51	2,105.69

bit widths at fixed (B, L) , the pooled slope of $\log T$ on $\log C_{\text{alloc}}$ is 0.14 for KIVI and -0.06 for KVQuant, against 0.30 and 0.24 if latency were proportional to modeled traffic; TurboQuant[†]’s slope of 1.24 mixes byte and kernel-variant effects.

3.3 Same-work decode latency

At all 357 points where BF16 and a compressed configuration were measured with identical work, the same-work ratio is below 1 (Figure 1). By family, S spans 0.471–0.842 for KIVI, 0.048–0.702 for TurboQuant[†], and 0.008–0.092 for KVQuant. Another 123 candidate pairs have no ratio: in 108 BF16 is capacity-infeasible, and 15 are adaptive contexts with no exact BF16 counterpart. Of the 108, the 27 with a feasible compressed configuration are the capacity points of Section 3.4; in the other 81 both sides are infeasible.

The families differ in shape (Table 4). KIVI, whose cache-path kernels are the official ones, is closest to BF16; it adds

about 7 ms at $B = 1, 4K$, most of it in the per-query-head kernels of our adapter (Section 4), and its ratio improves with batch size, peaking at 0.842 (k2v2). Removing KIVI’s whole fixed-cost excess in the decomposition of Section 4 ($\Delta c_0 = 5.0$ – 6.6 ms, which also contains FP16 staging and the official kernels’ fixed cost) gives the optimistic post-hoc bound $S_{\text{adj}} = T_{\text{BF16}} / (T_{\text{KIVI}} - \Delta c_0) \leq 1.005$ (k2v2, $B = 8, 24K$), an excess within the fit residual and the timing CV (0.35% each). KIVI thus at best matches BF16: k2v2 reads about 1/4.3 of BF16’s cache bytes, but its effective cache bandwidth is also about $4.4\times$ lower (379 against about 1,650 GB/s at the same batch size). TurboQuant[†]’s step time grows steeply with context but barely changes with batch at fixed L (TQ-4bit at 32K: 123.9 ms at $B = 1$, 122.7 ms at $B = 8$), and it is not monotone in bit width: TQ-k3v4 is the fastest of the three. Nsight traces locate the cause in one integration choice (Appendix D). The split-KV stage-1 decode kernel takes 90.6% of the traced GPU kernel time at $B = 1, 32K$. Our port fixes its split count at 4 where

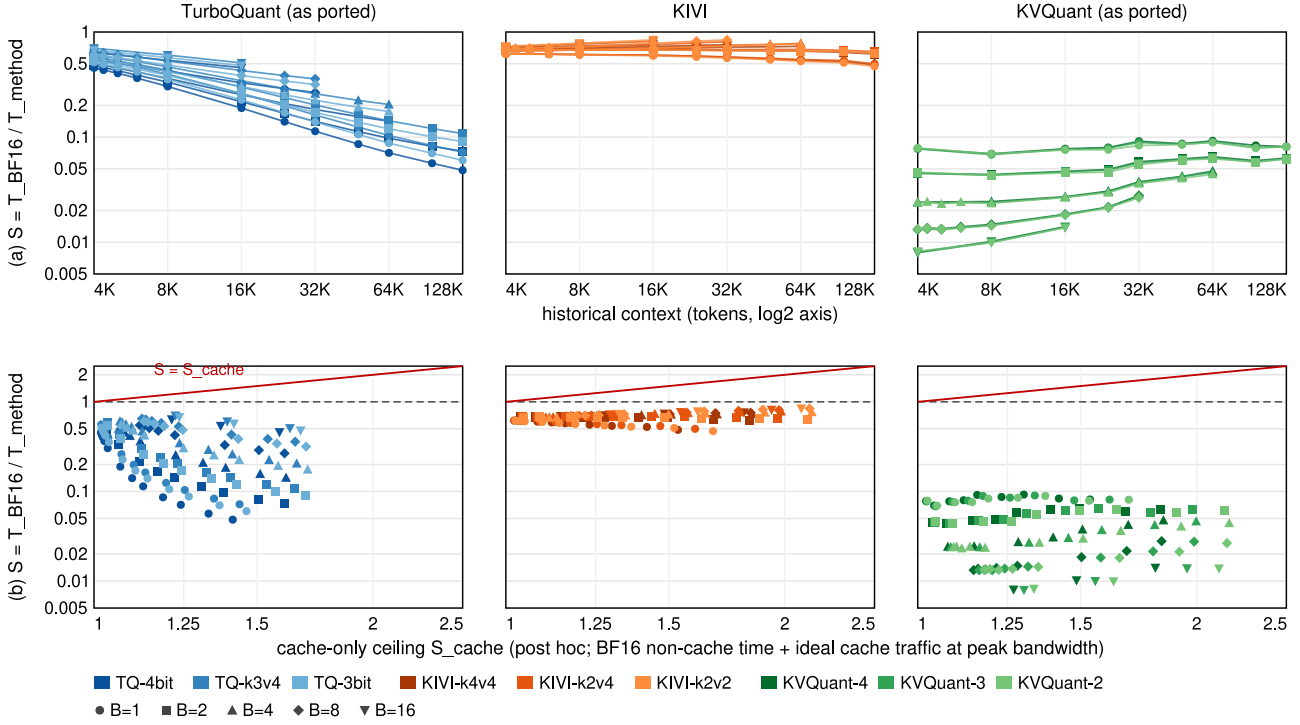


Figure 1. Wall-clock same-work ratio $S = T_{\text{BF16}}/T_{\text{method}}$ in Graph mode, one panel per compressed family, colors by configuration and markers by batch size; both axes logarithmic. (a) S versus historical context; every ratio is below 1 (dashed line). (b) The same 357 ratios against the post-hoc cache-only ceiling S_{cache} (Section 3.2); the red line marks a method running at its ceiling.

vLLM v0.25.1 uses 32, and it runs one warp per CTA, so its grid is $(B, 32, 4)$: 128 warps at $B = 1$, 1.4% of the GPU’s 9,024 resident-warp slots, and 1,024 one-warp CTAs at $B = 8$, still less than one wave. Additional sequences therefore fill idle SMs at little cost, while each context token lengthens every CTA’s work. The G1 reference fixtures were generated with the same split count, so the correctness gate could not detect the deviation. In a post-hoc diagnostic that changes only the split count to 32, stage-1 kernel time falls $7.3\text{--}7.9\times$ at $B = 1$ and $1.8\text{--}2.7\times$ at $B = 8$, yet TurboQuant stays $1.6\text{--}3.2\times$ slower than BF16 in traced GPU kernel time (Nsight Systems, not wall-clock benchmark timing) at the nine comparable points; all timings in this paper use the configuration as ported.

KVQuant costs about 150 ms already at $B = 1$, 4K and grows almost proportionally with batch (151, 1,078, and 2,154 ms at $B = 1$, 8, and 16), while BF16 goes from 11.8 to 17.3 ms over the same range. Most of this floor is `SelectFixedOutliers1024Cap12Kernel`, the graph-safe, deterministic outlier selector that our KVQuant patch introduced (Section 2.2). It is launched as `<<<rows, 1>>>` on one row at a time, so each launch runs a single GPU thread over a 1,024-wide row, about 2 ms per launch and 64 launches per sequence and step: 80.2%

of the traced GPU kernel time at $B = 1$, 24K. All three bit widths share this kernel, hence their common floor, and the adapter launches it, the key and value decode kernels, and the per-step packing kernels once per sequence on batch-1 slices, hence the near-proportional growth with B . Its time per step varies with context without a monotone trend (123–175 ms from 24K to 128K), for reasons we did not establish.

3.4 Capacity

Under the end-to-end peak used for feasibility, compression appears to buy capacity at the margin of this grid: at $B \cdot L = 393,216$ tokens ($B = 4$ at 96K, $B = 8$ at 48K, $B = 16$ at 24K), BF16’s predicted peak of 114.3–114.4 GB exceeds the 89.7 GB limit, while all nine compressed configurations fit at 72.1–86.0 GB. That boundary is set by prefix construction, not by the cache: the predicted peak includes a 40.3 GB MLP transient of the unchunked prefill. In post-hoc steady-state accounting from the same feasibility formula (weights, cache, decode workspace, and the CUDA Graph pool), BF16 needs 74.1 GB at those points and fits, and 505 rather than 441 of the 534 grid points are feasible. Points where only compressed configurations fit in steady state do exist (61; for example, TQ-4bit at $B = 4$,

128K needs 55.5 GB against BF16’s 93.4 GB), but none was measured, because the end-to-end peak excluded them. The 27 points thus carry no speedup claim, and this study makes no measured capacity claim; all memory figures here are predictions.

3.5 GQA materialization

Expanding K/V from 8 to 32 heads creates operands four times larger than the stored cache. Source audits, operator traces, allocation audits, and profiler kernel classification found such expansion only outside the validated system. Generic reference attention paths expand K/V: Transformers’ eager attention calls `repeat_kv`, and a same-shape math-SDPA control dispatched `expand` and `clone`, so we excluded both as the baseline. The residual-window attention of the official KIVI snapshot also produced a contiguous 32-head copy with four times the input storage at our geometry; our source patch replaces it with KV-head-grouped products, and the measured adapter reads the eight-head residual storage directly (Section 2.2). Every validated configuration passed the G3 materialization check, and at the profiled point Nsight Compute found no complete-prefix materialization or query-head-expanded K/V signature. Fourfold expansion is therefore a property of particular code paths, not of GQA decoding in general; we make no claim about how common it is elsewhere.

3.6 Launch overhead and CUDA Graphs

Fixed- L latency curves are often described by a floor, a knee, and a post-knee slope, and a natural hypothesis is that the floor is launch overhead that CUDA Graphs [24] remove. We tested this in a dedicated eager-versus-Graph experiment that compares the two modes within a configuration and shape. It covered all ten configurations at $B \in \{1, 4\}$ and six contexts from 4K to 128K, with three processes per mode; of 120 conditions, the 105 stable ones enter the fitted comparisons (Appendix A).

The preregistered pure launch-floor interpretation requires that under Graph mode the fitted floor drops materially (ratio ≥ 1.05), the post-knee slope stays within $[0.8, 1.2]$ of eager, a host-minus-device timing proxy decreases, and outputs and kernel paths are unchanged. It holds in 0 of 14 fully identifiable comparisons; four more are inconclusive because of unstable eager data, and two have no positive eager slope. The failures are method-specific (Appendix Figure A1). For KIVI, Graph mode lowers the fitted floor about tenfold (ratio 9.8–10.3) but also changes the slope (ratio 0.29–0.31 at $B = 1$) and moves the knee by $-61,440$ tokens. For TurboQuant[†], floors drop modestly (1.05–1.59) with similar slopes, but the timing proxy does not decrease. For KVQuant, neither floor nor slope changes materially.

Nsight Systems [26] shows what Graph mode removes for

four anchor configurations (BF16, TQ-4bit, KIVI-k4v4, KVQuant-4). In the traced window, eager execution issued 10,832 CUDA API calls for BF16, 12,400 for TurboQuant, 54,608–63,312 for KIVI, and 53,328 for KVQuant; Graph mode replaced them with eight graph launches. GPU inter-kernel idle time fell in all 16 anchor pairs, with kernel count and order unchanged. Graph mode thus removes submission overhead and idle gaps for every method, but how much that reshapes the latency curve depends on the method.

4 LATENCY DECOMPOSITION AND PREDICTIVE MODELING

A post-hoc decomposition describes the measured latency surfaces with three terms. At each batch size we fit, across contexts,

$$T(L) \approx c_0 + D_{\text{cache}}(L)/BW_{\text{eff}},$$

where D_{cache} is the modeled cache-path traffic (Appendix C), c_0 collects non-cache work and fixed overhead, and BW_{eff} is the effective cache-path bandwidth. The fit is descriptive: $R^2 \geq 0.998$ for BF16, TurboQuant, and KIVI at every batch size, and 0.75–0.998 for KVQuant. BF16 has $c_0 \approx 12$ ms, which covers the weights and other non-cache traffic (15.1 GB per step), and $BW_{\text{eff}} = 1.60$ – 1.66 TB/s (89–93% of peak) at every batch size. TurboQuant[†] has about the same c_0 , but its BW_{eff} grows in proportion to B , from 14 GB/s at $B = 1$ to 208 GB/s at $B = 16$ (TQ-4bit), as expected when each sequence’s work runs on a fixed, small set of CTAs. KIVI adds about 6 ms of fixed cost and reaches 0.14–0.23 TB/s at $B = 1$, rising to 0.38–0.48 TB/s (21–27% of peak) at $B = 8$ and 16. In the Graph-mode Nsight Systems trace of KIVI-k4v4 at $B = 1$, 4K (GPU kernel time, not wall-clock), its quantized-cache kernels take 0.84 ms per step, while 3,072 FP16 bmm and 2,048 FP16 add kernels, which our adapter issues per query head for the residual window and the current token, take 5.9 ms; BF16’s attention kernels take 0.66 ms. KVQuant’s fixed cost grows with B at 130–150 ms per sequence. In these implementations, cache bytes add latency at the rate $1/BW_{\text{eff}}$, and BW_{eff} is set by the kernels.

The preregistered predictive models reach the same conclusion from the other direction (Appendix E). A scalar byte law, quadratic in $\log(B \cdot L/r_{\text{alloc}})$, has a held-out macro median error of 34.6%. The selected method-conditioned knee surface in B , L , and r_{alloc} (model D) reaches 7.9% but misses all four evaluable targets: macro P95 error 164.5%, dominated by a BF16 batch-edge extrapolation failure; same-work sign accuracy 93.2%; and pairwise ranking accuracy 67.4%. It interpolates within held-out context bands (median error 1.8–9.7%) and is unvalidated outside the measured domain.

5 CACHE-SENSITIVE QUALITY EVALUATION AND SCOPED JOINT RESULTS

5.1 Protocol

Quality was evaluated after the performance data were frozen, on the same checkpoint, tokenizer, and method configurations. Every stage runs at physical batch size 1 in an eager, growing-context path that forces the model to read its compressed cache. Perplexity uses incremental teacher forcing after a prefix, so a decode step over the method’s cache predicts every scored token. LongBench prompts prefill all but the final 16 prompt tokens, decode those 16 through the method, and then generate greedily. Token IDs, prompts, and the 16-token split are identical for BF16 and every method. All comparisons are paired, with 10,000 bootstrap draws clustered by document for perplexity and stratified by task (LongBench-E) or by length and category (LongBench v2).

Stages run in a fixed order, and a configuration advances only if it passes the previous stage: correctness checks; Fast perplexity; Full perplexity; LongBench-E; LongBench v2 without chain of thought (CoT) for per-method finalists; a CoT stress test for final recommended configurations; and joint validation. A secondary “native prefill” protocol, which prefills the whole prompt at once, applies to finalists only. Margins were fixed before any quality output: for perplexity, the paired 95% upper bound on ΔNLL stays below $\log(1.01)$ per dataset, with per-length review and hard-fail thresholds of 2% and 5%; for LongBench-E, the macro-drop upper bound is at most 2 points, with category thresholds of 3 (review) and 5 (hard) points and an invalid-output increase of at most 1 percentage point (pp). The LongBench v2 rules appear in Table 6.

Datasets are WikiText-2 test and a fixed C4 validation subset of 2,048 documents drawn from the first 4,096 rows [23, 29]; LongBench-E with 13 tasks and 3,668 samples [3]; and LongBench v2 with 503 frozen IDs, 321 of which fit the model’s length under the no-CoT budget [4].

5.2 Protocol history

The quality protocol changed in documented steps. Its stages, gates, and margins were written before any performance measurement. The exact inputs (dataset revisions, sample IDs, prompt and chat-template hashes) and several mechanical choices (bootstrap size and resampling units, the bounded C4 frame, length buckets, the CoT subset and budgets) were fixed after performance results were known and before any quality output. The original protocol made batch invariance ($B = 1$ versus $B \in \{4, 8\}$) a correctness gate, and all ten configurations, BF16 included, failed it (Section 5.3). After that failure and before any perplexity score

Table 5. Perplexity versus BF16. Fast: 128 anchors and 16,384 scored tokens per configuration, at prefixes of 4K, 24K, 32K, and 64K. Full: 1,024 anchors and 262,144 scored tokens per configuration, at eight prefixes from 4K to 130,560. Intervals are paired 95% CIs on ΔNLL . KVQuant rows are in Appendix F.

Configuration	Fast ΔNLL [95% CI]	Fast rel. PPL	Fast status	Full status
TQ-4bit	0.0103 [0.0078, 0.0128]	+1.03%	INCONCLUSIVE	not run
TQ-k3v4	0.0339 [0.0289, 0.0392]	+3.45%	FAIL	not run
TQ-3bit	0.0389 [0.0337, 0.0442]	+3.97%	FAIL	not run
KIVI-k4v4	0.0016 [0.0005, 0.0028]	+0.17%	PASS	PASS
KIVI-k2v4	0.0367 [0.0316, 0.0421]	+3.74%	FAIL	not run
KIVI-k2v2	0.0434 [0.0376, 0.0494]	+4.43%	FAIL	not run

was computed, the protocol was amended to restrict formal quality evaluation to physical $B = 1$. The amendment records the cross-batch gate as failed, keeps every margin, dataset, sample, tokenization, and statistic, and states that $B = 1$ quality does not transfer to $B > 1$. Appendix B gives dates.

5.3 Correctness and batch-shape sensitivity

All ten configurations pass seven growing-context probes ($L = 512$ to 130,560), the 16-token LongBench suffix check, finite-output checks, and eager/Graph invariance at the fixed- L shape (100 of 100 tokens, zero maximum logit difference). The nine compressed configurations also pass cache-dependence controls: perturbing encoded storage changes the output, and intercepting the decode path stops the run.

All ten fail batch invariance, with maximum absolute logit errors from 0.34 (BF16, $B = 4$) to 3.03 (KIVI-k2v2, $B = 4$) against a tolerance of 0.02 absolute plus 0.02 relative. For BF16, a diagnosis found identical inputs and embeddings and identical outputs for duplicated rows within a batch; the first difference appears at the layer-0 query or value projection, pointing to shape-dependent BF16 projection arithmetic, a known property of GEMM kernels that are not batch-invariant [14]. The compressed configurations were not diagnosed separately. All configurations and the performance scan share these projections, so outputs at $B > 1$ can differ numerically from $B = 1$. The timings at $B > 1$ remain valid measurements of these paths; only the quality results are limited to $B = 1$.

5.4 Perplexity

KIVI-k4v4 passes Fast perplexity and then passes Full perplexity on both datasets: +0.174% on WikiText-2 (ΔNLL

Table 6. LongBench v2 (no CoT, 8-token answer budget), BF16 versus KIVI-k4v4 on 321 paired questions. Drops are in percentage points.

Subset	N	BF16 correct	k4v4 correct	Drop [95% CI]
Overall	321	94 (29.3%)	89 (27.7%)	1.56 [−0.31, 3.43]
Single-document QA	126	43	40	2.38 [−0.79, 6.35]
Multi-document QA	90	29	28	1.11 [0.00, 3.33]
Long in-context learning	41	10	10	0.00 [−9.76, 9.76]
Long-dialogue history	39	4	4	0.00 [0.00, 0.00]
Code repository understanding	15	7	6	6.67 [0.00, 20.00]
Long structured data	10	1	1	0.00 [0.00, 0.00]

Criterion	Observed	Rule	Result
Overall non-inferiority	upper bound 3.43 pp	≤ 2 pp	INCONCLUSIVE
Category guardrail	code: 6.67 pp, $N = 15$	every category ≤ 5 pp	FAIL
Length guardrail	max 3.57 pp (8–16K, $N = 28$)	every bucket ≤ 5 pp	PASS
Invalid-output increase	+0.31 pp (24 $\rightarrow$ 25)	≤ 1 pp	PASS
BF16-correct retention	86/94 = 91.5%, CI [85.6%, 96.6%]	point estimate $\geq 95\%$	FAIL

0.00174, CI [0.00123, 0.00224]) and +0.182% on C4 (0.00182, [0.00115, 0.00257]). No dataset-length cell crosses the review threshold; the largest is +0.35%. The aggregate interval for TQ-4bit straddles the 1% margin (relative PPL CI [+0.78%, +1.29%]), so its Fast status is INCONCLUSIVE; no extension sample was run and it did not advance. The other seven configurations fail.

The three KVQuant configurations fail with Δ NLL near 3.4 at every bit width (Appendix F). A degradation that does not shrink from 2 to 4 bits is inconsistent with quantization error, so we treat this result post hoc as invalid, likely caused by a defect in our port, and exclude KVQuant from the quality conclusions; Table 7 keeps its preregistered status, and the exclusion is recorded with a machine-readable reason.

5.5 LongBench-E

On 3,668 paired samples, the equal-task macro score is 54.381 for BF16 and 54.249 for KIVI-k4v4: a drop of 0.132 points with paired 95% CI [−0.291, 0.545], well inside the 2-point margin. Category drops range from −0.73 to +0.78 points, length-bucket drops from 0.02 to 0.22 points, and neither method produced an invalid output. KIVI-k4v4 passes LongBench-E.

5.6 LongBench v2

The overall interval includes zero but extends past the 2-pp margin, so overall non-inferiority is inconclusive. The combined gate fails on two guardrails, both fixed rules applied to small counts; the estimated overall drop is 1.56 pp. The code-category failure is a single-answer flip: of $N = 15$ code repository questions, BF16 answers 7 correctly and KIVI-k4v4 answers 6, a 6.67-pp drop with 95% CI [0.0, 20.0] pp. The retention rule compares a point estimate

with 95%: KIVI-k4v4 retains 86 of the 94 questions BF16 answers correctly (91.5%), although the interval [85.6%, 96.6%] includes 95% ($n_{11}/n_{10}/n_{01}/n_{00} = 86/8/3/224$). The benchmark is also hard for this model under this budget: BF16 scores 29.3%, against 25% for chance on four-option questions; 253 of its 321 answers hit the 8-token budget (256 for KIVI-k4v4); and 24 and 25 outputs were invalid, including 20 of 39 per method in the long-dialogue category. We ran no null calibration, such as a BF16-versus-BF16 comparison, so the false-failure rate of these guardrails is unknown.

5.7 Joint outcome

Over the six TurboQuant and KIVI configurations, five fail and one (TQ-4bit) is inconclusive; KIVI-k4v4 is the only configuration that passes perplexity and LongBench-E, and none is fully qualified under the evaluation protocol. The KVQuant row of Table 7 keeps its preregistered status but is excluded from this statement (Section 5.4). The CoT stress stage and the native-prefill secondary evaluation were not run, since no finalist remained and the protocol applies both only to finalists. In the joined quality–performance table, the 600 planned runs at $B = 1$ carry configuration-level quality status, while the 2,070 planned runs at $B > 1$ lie outside the evaluated quality scope and carry no quality verdict (Appendix A). The qualified table is empty, so the best qualified configuration and its speedup are undefined; independently of quality, no same-work ratio exceeds 1.

6 LIMITATIONS AND CONCLUSIONS

6.1 Limitations

Scope. One model, one GPU, one software stack, tensor parallelism 1, static batches, and fixed- L Graph decode. We

Table 7. Joint outcome at physical $B = 1$. “Not run” marks stages a configuration never reached because an earlier stage did not pass. [‡]Post hoc: treated as invalid because of a suspected port defect and excluded from the quality conclusions (Section 5.4); the preregistered status is shown unchanged.

Configuration	Fast PPL	Full PPL	LongBench-E	LongBench v2	Outcome
TQ-4bit	INCONCLUSIVE	not run	not run	not run	inconclusive
TQ-k3v4, TQ-3bit	FAIL	not run	not run	not run	fail
KIVI-k4v4	PASS	PASS	PASS	FAIL	fail
KIVI-k2v4, KIVI-k2v2	FAIL	not run	not run	not run	fail
KVQuant-4, -3, -2 [‡]	FAIL	not run	not run	not run	fail

do not measure prefill cost, continuous batching, queuing, multi-GPU execution, or a serving system. The RTX PRO 6000 is a GB202 (sm_120) GPU with GDDR7, not a datacenter sm_100 Blackwell part with HBM, and kernel maturity differs between the two.

Implementations and positive control. All compressed families run adapted code, and the latencies characterize these implementations on this GPU. We measured no positive control—no production-grade quantized-KV kernel such as the FP8 KV caches of vLLM or TensorRT-LLM—so we cannot exclude that optimized kernels for the same algorithms would be faster; the modeled cache-only ceiling S_{cache} (1.02–2.17 at the same-work points) bounds what faster cache kernels could gain, given the modeled traffic. TurboQuant runs as ported with four KV splits where vLLM uses 32 (with 32 splits its GPU kernel time is still 1.6–3.2 $\times$ BF16’s in the post-hoc Nsight Systems diagnostic, not wall-clock time), KVQuant’s latency is dominated by our single-threaded selection kernel and per-sequence dispatch (Section 3.3), and most of KIVI’s fixed cost of about 6 ms comes from the per-query-head kernels of our adapter (Section 4); a rescan with the upstream split count and an FP8 KV-cache positive control are the next steps.

Post-hoc analyses. The effective bandwidths, ceilings, elasticities, steady-state capacity, decomposition, and Nsight diagnostics were defined after the results were known (Appendices C and D). Nsight Compute measured traffic with cold caches and base clocks at $B = 1$ for all configurations and at $B = 8$ and 16 for three of them (BF16, TQ-4bit, KIVI-k4v4); traffic elsewhere is modeled, and profiler kernel times appear only as shares and ratios. Memory figures are predictions of the feasibility formula, and the steady-state capacity gains of Section 3.4 were not measured.

Quality. All quality evidence is at physical $B = 1$. Quality ran in an eager, growing-context path linked to the Graph fixed- L performance runs through configuration identity and exact-shape eager/Graph agreement. LongBench v2 used no CoT and an 8-token budget; subgroups are small and three subgroup retention intervals are undefined; the C4 subset comes from a bounded frame; only one compressed configuration reached LongBench; and KVQuant’s quality

result is excluded as invalid.

6.2 Conclusions

On one Blackwell workstation GPU running a full Llama-3.1-8B decode step, the three KV-cache quantizers we measured move fewer bytes than BF16, but in these implementations fewer bytes do not mean lower latency: every compressed configuration, TurboQuant[†] included, is slower than BF16 FlashAttention at every comparable point. The cleanest comparison, KIVI, whose cache-path kernels are the official ones, reads 2.8–4.3 $\times$ fewer cache bytes yet runs its cache path at 8–13% of peak DRAM bandwidth against BF16’s 89% at batch size 1, and reaches at most 0.842 of BF16’s speed; even with its whole fixed-cost excess removed it at best matches BF16 ($S_{\text{adj}} \leq 1.005$, post hoc), because its effective bandwidth falls about as much as its bytes. In all three implementations, large and identifiable costs trace to integration choices rather than to the algorithms—four KV splits where vLLM uses 32, a single-threaded outlier selector, and per-query-head residual kernels in our KIVI adapter; with 32 splits, TurboQuant’s GPU kernel time is still 1.6–3.2 $\times$ BF16’s (Nsight Systems diagnostic, not wall-clock). A fixed cost plus cache bytes over an effective bandwidth describes the latency surfaces, so bytes matter only through a bandwidth that the kernels set. Capacity gains appear only once prefix-construction transients are excluded, and none was measured. At batch size 1, KIVI k4v4 passes perplexity and LongBench-E before failing two small-count LongBench v2 guardrails, and KVQuant’s quality result is invalid. Our correctness gates passed every configuration yet caught none of these performance-relevant integration choices; admission should also check configuration parity with upstream, and that the code a paper describes is the code that was measured. Each link—bit width, allocated bytes, measured traffic, same-work latency, and cache-sensitive quality—needs its own measurement. The released CPU packages regenerate the frozen tables; the post-hoc analyses and the raw data are available on request (Appendix A).

REFERENCES

- [1] Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. arXiv:2305.13245v3, 2023.
- [2] Ashkboos, S., Mohtashami, A., Croci, M. L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoeffer, T., and Hensman, J. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. arXiv:2404.00456v2, 2024.
- [3] Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and Li, J. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. arXiv:2308.14508v2, 2023.
- [4] Bai, Y., Tu, S., Zhang, J., Peng, H., Wang, X., Lv, X., Cao, S., Xu, J., Hou, L., Dong, Y., Tang, J., and Li, J. LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks. arXiv:2412.15204v2, 2024.
- [5] Cai, Z., Zhang, Y., Gao, B., Liu, Y., Li, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Hu, J., and Xiao, W. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. arXiv:2406.02069v4, 2024.
- [6] Chang, C.-C., Lin, W.-C., Lin, C.-Y., Chen, C.-Y., Hu, Y.-F., Wang, P.-S., Huang, N.-C., Ceze, L., Abdelfattah, M. S., and Wu, K.-C. Palu: KV-Cache Compression with Low-Rank Projection. arXiv:2407.21118v2, 2024.
- [7] Chari, V. and Van Durme, B. Compactor: Calibrated Query-Agnostic KV Cache Compression with Approximate Leverage Scores. arXiv:2507.08143v2, 2025.
- [8] Dao, T. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691v1, 2023.
- [9] Devoto, A., Jeblick, M., and Jégou, S. Expected Attention: KV Cache Compression by Estimating Attention from Future Queries Distribution. arXiv:2510.00636v1, 2025.
- [10] Duanmu, H., Yuan, Z., Li, X., Duan, J., Zhang, X., and Lin, D. SKVQ: Sliding-window Key and Value Cache Quantization for Large Language Models. arXiv:2405.06219v3, 2024.
- [11] Feng, Y., Indyk, P., Kapralov, M., Krachun, D., and Prokhorov, B. Provable Quantization with Randomized Hadamard Transform. arXiv:2605.13810v1, 2026.
- [12] Gao, W., Zhou, X., Sun, P., Zhang, T., and Wen, Y. Rethinking Key-Value Cache Compression Techniques for Large Language Model Serving. arXiv:2503.24000v1, 2025.
- [13] Grattafiori, A. et al. The Llama 3 Herd of Models. arXiv:2407.21783v3, 2024.
- [14] He, H. and Thinking Machines Lab. Defeating Nondeterminism in LLM Inference. Thinking Machines Lab: Connectionism, <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>, 2025.
- [15] He, Y., Zhang, L., Wu, W., Liu, J., Zhou, H., and Zhuang, B. ZipCache: Accurate and Efficient KV Cache Quantization with Salient Token Identification. arXiv:2405.14256v1, 2024.
- [16] Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. arXiv:2401.18079v6, 2024.
- [17] Kang, H., Zhang, Q., Kundu, S., Jeong, G., Liu, Z., Krishna, T., and Zhao, T. GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. arXiv:2403.05527v4, 2024.
- [18] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180v1, 2023. Software: vLLM v0.25.1, <https://github.com/vllm-project/vllm>.
- [19] Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. SnapKV: LLM Knows What You are Looking for Before Generation. arXiv:2404.14469v2, 2024.
- [20] Liu, Y., Li, H., Cheng, Y., Ray, S., Huang, Y., Zhang, Q., Du, K., Yao, J., Lu, S., Ananthanarayanan, G., Maire, M., Hoffmann, H., Holtzman, A., and Jiang, J. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. arXiv:2310.07240v6, 2023.
- [21] Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. arXiv:2402.02750v2, 2024.
- [22] Liu, Z., Ma, X., Luo, D., Zhao, H., Lu, B., Huang, W., Gu, Y., Liu, X., Wei, Z., Liu, J., Tao, D., and

- Tan, G. KVServe: Service-Aware KV Cache Compression for Communication-Efficient Disaggregated LLM Serving. arXiv:2605.13734v1, 2026.
- [23] Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer Sentinel Mixture Models. arXiv:1609.07843v1, 2016.
- [24] NVIDIA. CUDA Programming Guide, Section 4.2: CUDA Graphs. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cuda-graphs.html> (accessed 2026-10-04), 2026.
- [25] NVIDIA. Nsight Compute Documentation. <https://docs.nvidia.com/nsight-compute/> (accessed 2026-10-04), 2026.
- [26] NVIDIA. Nsight Systems Documentation. <https://docs.nvidia.com/nsight-systems/> (accessed 2026-10-04), 2026.
- [27] NVIDIA. Quantization. TensorRT-LLM documentation, <https://nvidia.github.io/TensorRT-LLM/latest/features/quantization.html> (accessed 2026-10-05), 2026.
- [28] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. PyTorch: An Imperative Style, High-Performance Deep Learning Library. arXiv:1912.01703v1, 2019.
- [29] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. arXiv:1910.10683v4, 2019.
- [30] Tayaranian Hosseini, S. M., Ardakani, A., and Gross, W. J. InnerQ: Hardware-Aware Tuning-Free Quantization of KV Cache for Large Language Models. arXiv:2602.23200v2, 2026.
- [31] vLLM Project. Quantized KV Cache. vLLM documentation, https://docs.vllm.ai/en/latest/features/quantization/quantized_kvcache.html (accessed 2026-10-05), 2026.
- [32] Wang, H., Han, L., Xu, K., and Srivastava, A. SQuat: Subspace-orthogonal KV Cache Quantization. arXiv:2503.24358v2, 2025.
- [33] Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient Streaming Language Models with Attention Sinks. arXiv:2309.17453v4, 2023.
- [34] Xiao, G., Tang, J., Zuo, J., Guo, J., Yang, S., Tang, H., Fu, Y., and Han, S. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. arXiv:2410.10819v1, 2024.
- [35] Yang, J. Y., Kim, B., Bae, J., Kwon, B., Park, G., Yang, E., Kwon, S. J., and Lee, D. No Token Left Behind: Reliable KV Cache Compression via Importance-Aware Mixed Precision Quantization. arXiv:2402.18096v1, 2024.
- [36] Zandieh, A., Daliri, M., and Han, I. QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead. arXiv:2406.03482v2, 2024.
- [37] Zandieh, A., Daliri, M., Hadian, M., and Mirrokni, V. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. arXiv:2504.19874v1, 2025.
- [38] Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., and Chen, B. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. arXiv:2306.14048v3, 2023.